RutaIA

TURISMO INTELIGENTE · GUATEMALA

PROYECTO FINAL · PROGRAMACIÓN COMERCIAL

Turismo con IA, agentes multi-paso y control de tokens

Chatbot RAG, dos agentes, cuotas y analítica en Power BI

React + Vite · Express · PostgreSQL 16 · Groq · Power BI
https://rutaia-api-q3y2.onrender.com
Swagger: https://rutaia-api-q3y2.onrender.com/api/docs
Repositorio: github.com/FerMar707501/PFinalTurismo

Entrega académica · Noviembre de 2026

1. Propuesta y alcance

RutaIA es una aplicación web de turismo para **Guatemala** que combina un catálogo administrable con tres servicios de inteligencia artificial: un chatbot con generación aumentada por recuperación (RAG), un agente planificador de itinerarios y un agente comparador de destinos. La plataforma autentica por roles, limita el consumo mensual de tokens, registra cada ejecución y expone un reporte analítico compartido por el panel web y Power BI.

Problema. Planificar un viaje por Guatemala exige consultar fuentes dispersas: clima por región, distancias reales entre destinos, actividades disponibles y precios en quetzales. Un asistente conversacional que consulte datos verificables reduce ese trabajo y, además, permite medir el costo real de cada respuesta en tokens.

Alcance. Turismo exclusivamente en Guatemala: 12 destinos reales, 48 actividades, precios en quetzales (GTQ) y validación de coordenadas contra el *bounding box* del país. Quedan fuera del alcance pagos, reservas reales, mapas interactivos y aplicaciones móviles nativas.

- **Chat RAG:** recuperación léxica sobre 60 documentos, memoria de la conversación y fuentes citadas.
- **Agentes multi-paso:** *tool calling* con Groq, hasta ocho pasos, herramientas internas y externas.
- **Control de cuota:** estimación previa, bloqueo con HTTP 429 y registro del consumo real posterior.
- **Administración:** usuarios y cuotas, destinos, actividades, servicios de IA y claves de API.
- **Analítica:** un mismo agregado para la web y Power BI, con exportación CSV.
- **Calidad:** 75 pruebas con Vitest, Swagger, colección Postman y CI/CD en GitHub Actions.

Requisitos del curso y su implementación

Requisito del curso	Implementación en RutaIA	Evidencia
Chatbot con RAG y memoria	POST /api/chat con recuperación de 5 documentos y últimos mensajes	Código, pruebas Fase 4 y captura 04
Dos servicios de IA adicionales	Agentes de itinerario y comparación con hasta 6 pasos y 5 herramientas	Pruebas Fase 5 y capturas 05-06
Login con roles	JWT, bcrypt y roles admin/user; sin registro público	Pruebas Fase 3 y captura 01
CRUD administrativo	Usuarios, destinos, actividades y servicios de IA	Capturas 10-13
Límite de tokens	Cuota mensual, reserva previa, bloqueo 429 y registro real	Fases 4-5 y captura 08
Conversaciones e historial	Conversaciones por usuario, mensajes, fuentes y ejecuciones de agentes	Capturas 04 y 07
API REST con permisos	Rutas documentadas en OpenAPI 3.0.3, JWT, API keys y CORS	Swagger y Postman
Power BI conectado	Endpoint externo /api/bi/usage con X-API-Key y el mismo agregado	powerbi/README y captura 18
Pruebas automatizadas	75 pruebas con Vitest y Supertest	CI en verde
Despliegue y CI/CD	Render y GitHub Actions con smoke test	Workflows ci.yml, smoke.yml y CodeQL

2. Arquitectura de la solución



Un único servicio web en Render sirve la SPA compilada y la API desde el mismo dominio, lo que evita configurar CORS entre dos despliegues y simplifica los enlaces de entrega. La base de datos es un PostgreSQL gestionado por Render.

Capa	Tecnología	Responsabilidad
Frontend	React 19 + Vite	SPA con navegación por hash, roles, chat, agentes, administración y tema claro/oscuro
API	Express 5 (Node 22)	Rutas REST, validación Zod, JWT, cuotas, Swagger y OpenAPI
Datos	PostgreSQL 16	Catálogo, usuarios, conversaciones, uso, trazas y caché externa
IA	Groq (gpt-oss-120b / 20b)	Chat con contexto, decisiones multi-paso y tool calling
Fuentes abiertas	Open-Meteo · OSRM · Wikipedia	Clima, rutas y contexto cultural sin claves
BI	Power BI Desktop	Reporte de consumo con API key y refresco programado
Despliegue	Render · GitHub Actions	CI con pruebas, CodeQL y despliegue automático

3. Modelo de datos

El esquema usa **PostgreSQL 16** con codificación UTF-8 y almacenamiento en UTC. Aplica claves foráneas para preservar la trazabilidad y disparadores *updated_at*. El DDL completo está en `db/schema.postgres.sql` y los datos de demostración en `db/seed.postgres.sql`.

Tabla	Propósito	Claves y relaciones
users	Usuarios, rol y cuota mensual	PK id · UQ email
destinations	Catálogo turístico de Guatemala	PK id · coordenadas validadas
activities	Oferta de cada destino	FK destination_id → destinations
documents	Fragmentos indexados con tsvector para el RAG	source_type / source_id
ai_services	Modelo, temperatura, tokens y prompt	PK code
conversations / messages	Historial del chat y de los agentes	FK user_id y conversation_id
usage_logs	Hechos de consumo para cuotas y analítica	FK user_id, service_code, conversation_id
agent_runs	Entrada, salida, pasos y tokens por ejecución	FK user_id · service_code
api_clients	Claves de API con hash SHA-256 y scope	PK id · UQ key_hash
external_cache	Caché de clima, rutas y Wikipedia	PK cache_key · expires_at

Los datos de demostración contienen 3 usuarios, 12 destinos, 48 actividades, 3 servicios de IA y la clave de Power BI. La ingesta del RAG genera adicionalmente 60 documentos a partir del catálogo.

Creación de la base de datos

Con PostgreSQL 16 en ejecución y `DATABASE_URL` en `server/.env`:

```
cd server
npm install
npm run db:setup -- --force   # aplica schema.postgres.sql + seed.postgres.sql
npm run db:ingest             # genera los 60 documentos del RAG
```

Verificación: `SELECT COUNT(*) FROM destinations;` debe devolver 12, `activities` 48 y `documents` 60.

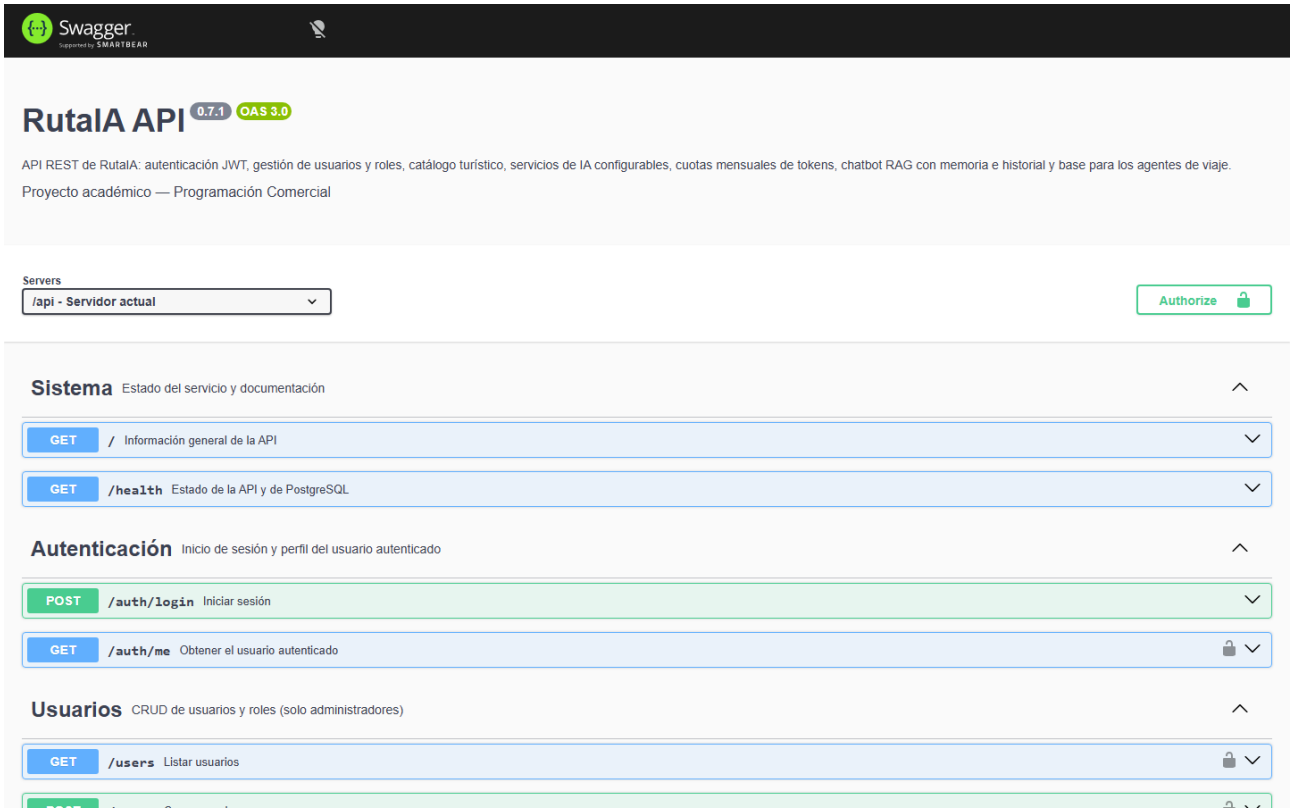
4. API REST y documentación

La API está documentada en OpenAPI 3.0.3 y disponible en **https://rutaia-api-q3y2.onrender.com/api/docs**. La autenticación principal es JWT; los consumidores externos usan una API key con scope `analytics:read`.

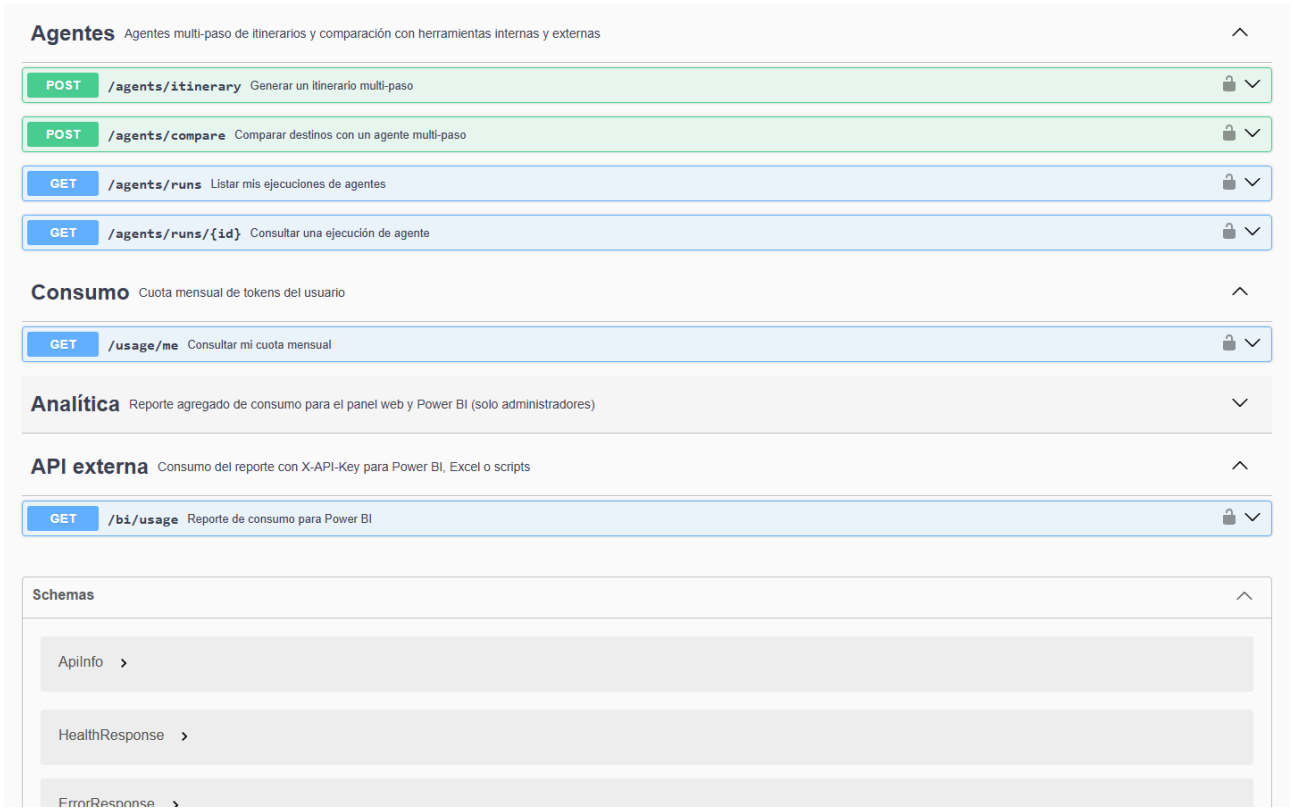
Método y ruta	Autenticación	Descripción
POST /api/auth/login	Pública	Entrega JWT y cuota del usuario
GET /api/auth/me	JWT	Perfil y cuota vigente
GET/POST /api/users	JWT admin	Listado y creación de usuarios
PATCH/DELETE /api/users/:id	JWT admin	Edición y baja lógica
GET /api/destinations, /:id	Pública	Catálogo y ficha con actividades
POST/PATCH/DELETE /api/destinations	JWT admin	CRUD y sincronización del RAG
GET /api/activities	Pública	Actividades con filtros
POST/PATCH/DELETE /api/activities	JWT admin	CRUD y sincronización del RAG
GET /api/services	Pública/JWT	Servicios activos; admin ve todos
PATCH /api/services/:code(/status)	JWT admin	Configuración y activación
GET /api/usage/me	JWT	Cuota mensual del usuario
POST /api/chat	JWT	Mensaje al chatbot RAG
GET /api/chat/conversations(/:id)	JWT	Historial y fuentes propias
POST /api/agents/itinerary	JWT	Agente de itinerarios
POST /api/agents/compare	JWT	Agente comparador
GET /api/agents/runs(/:id)	JWT	Trazabilidad de ejecuciones
GET /api/analytics/usage(.csv)	JWT admin	Reporte agregado y exportación
GET/POST/PATCH /api/analytics/clients	JWT admin	Gestión de API keys
GET /api/bi/usage	X-API-Key	Reporte para Power BI

Códigos de error relevantes

HTTP	Código	Cuándo ocurre
400	VALIDATION_ERROR	Datos inválidos, rango de fechas incorrecto o coordenadas fuera de Guatemala
401	INVALID_TOKEN / API_KEY_REQUIRED	JWT inválido o falta X-API-Key
403	FORBIDDEN / USER_INACTIVE	Sin rol admin o cuenta desactivada
409	SERVICE_DISABLED / EMAIL_IN_USE	Servicio apagado o correo repetido
429	TOKEN_QUOTA_EXCEEDED	La reserva de tokens supera la cuota
503	AI_NOT_CONFIGURED	No está configurada la clave de Groq



Swagger UI con los grupos de la API

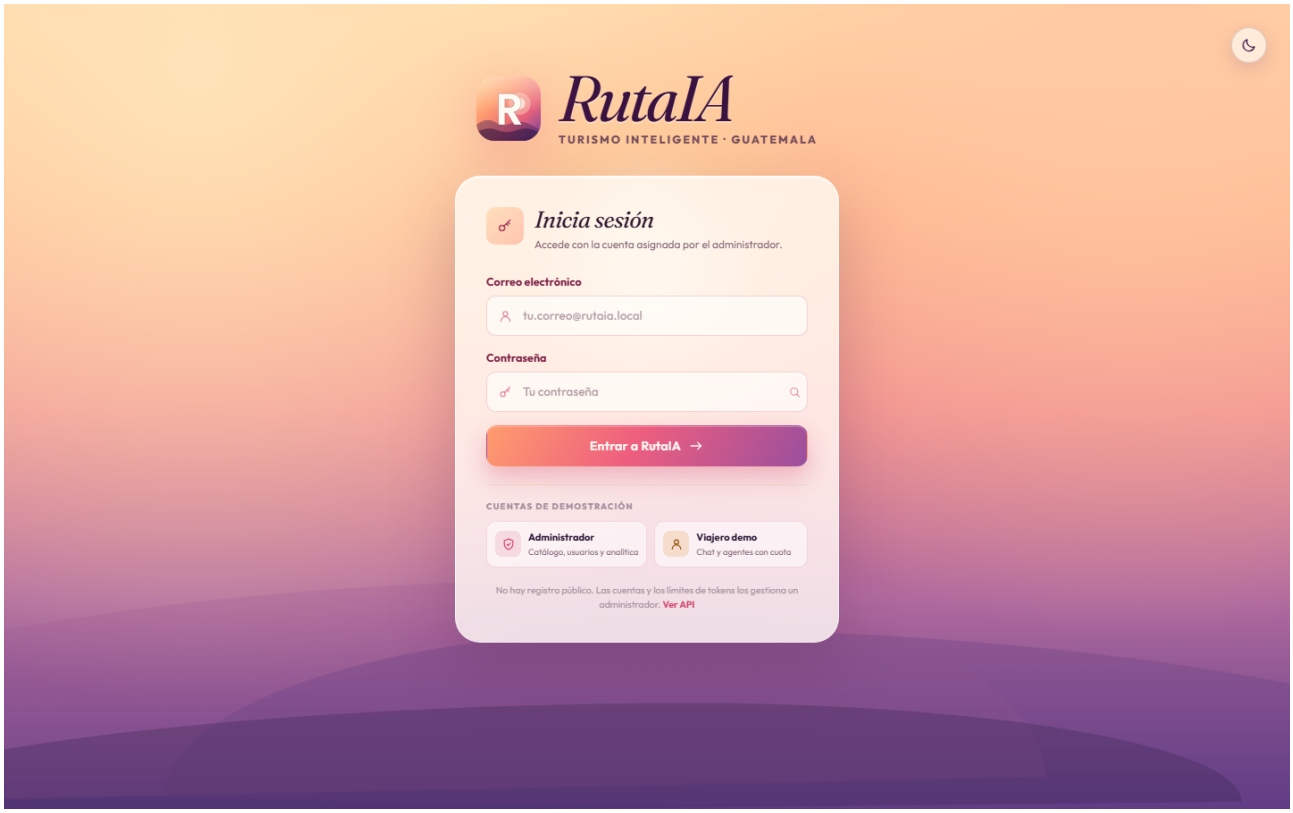


Endpoints de analítica y API externa en Swagger

5. Aplicación web

El frontend se construyó con React 19 y Vite, sin librerías adicionales de UI ni de Markdown. La navegación es por hash, el estado de sesión se guarda en `localStorage` y cada llamada envía el JWT. El diseño es responsive desde 320 px e incluye **tema claro y oscuro** con preferencia persistente y detección del tema del sistema.

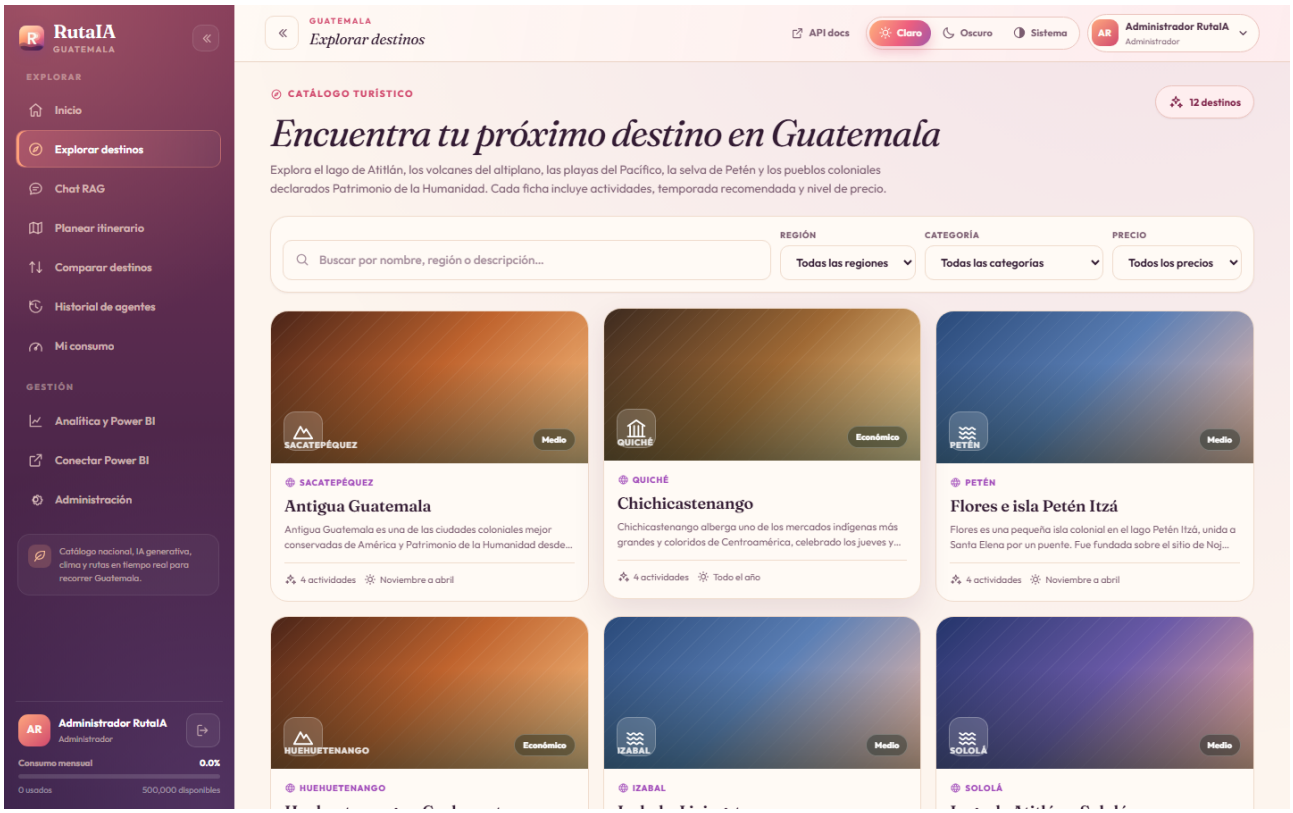
Identidad visual. La interfaz usa una paleta de atardecer (ámbar, coral, magenta y violeta) sobre fondos cálidos, con tipografía Outfit para el texto y Fraunces en los títulos. El logotipo es un SVG propio con degradado, sol y cordillera, reutilizado en el login, la barra lateral y la pantalla de carga.



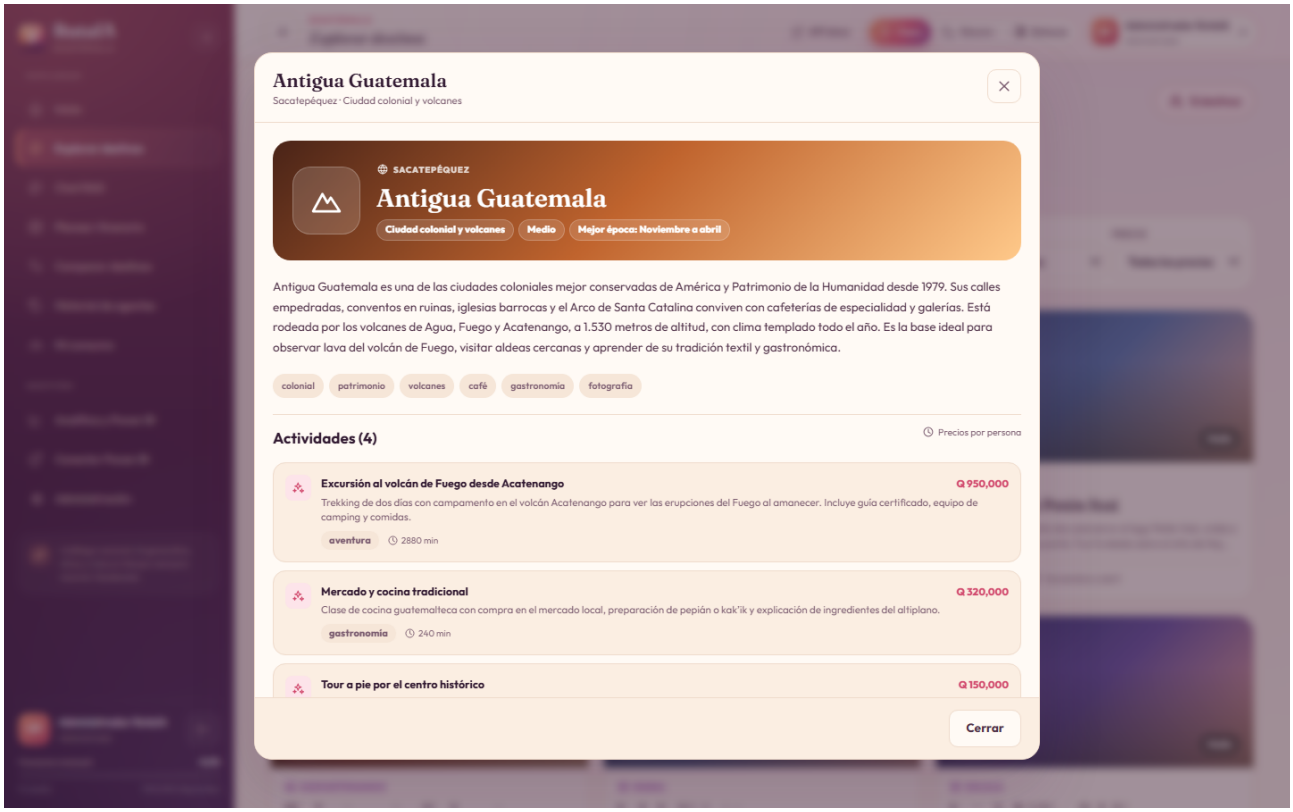
Inicio de sesión con el diseño de atardecer y cuentas de demostración



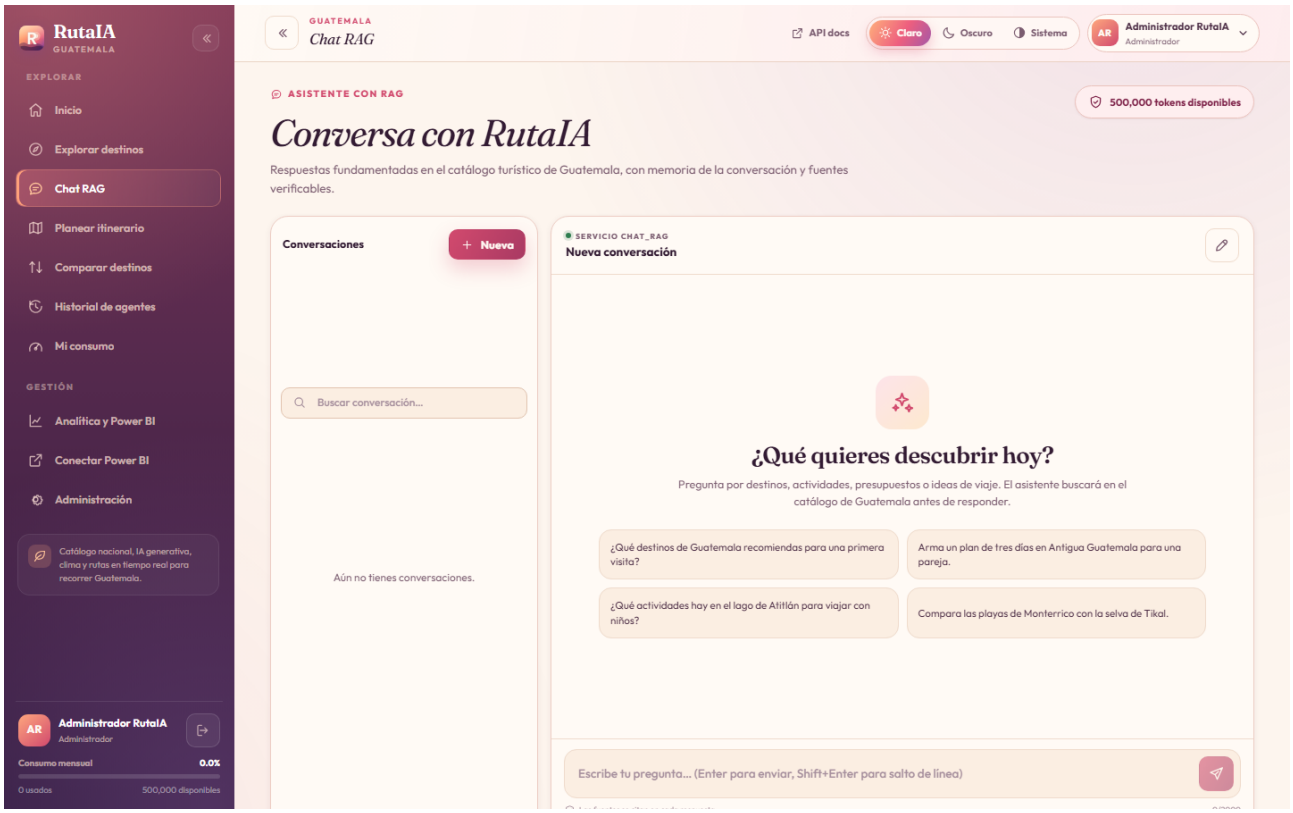
Panel del administrador: catálogo, servicios, cuota y accesos rápidos



Catálogo de destinos con búsqueda y filtros



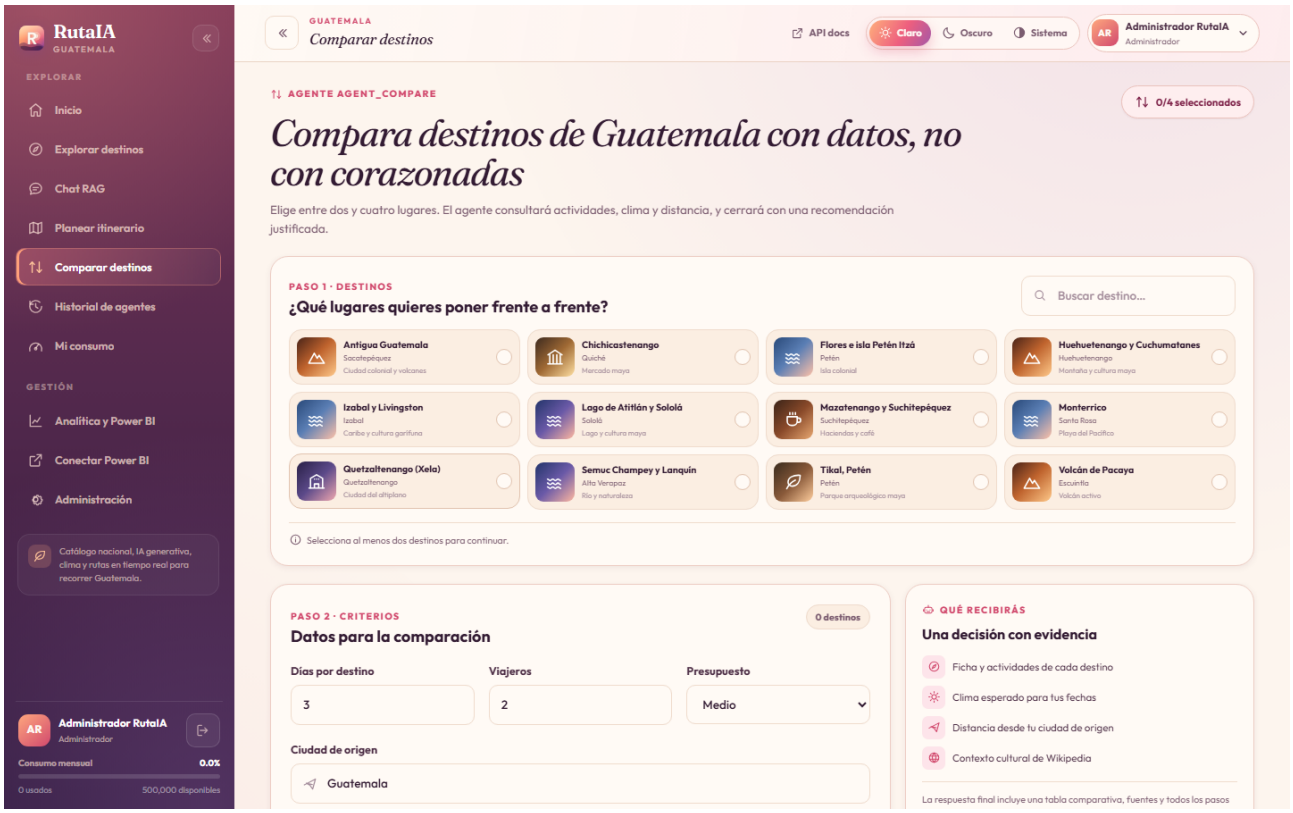
Ficha de destino con actividades y precios en quetzales



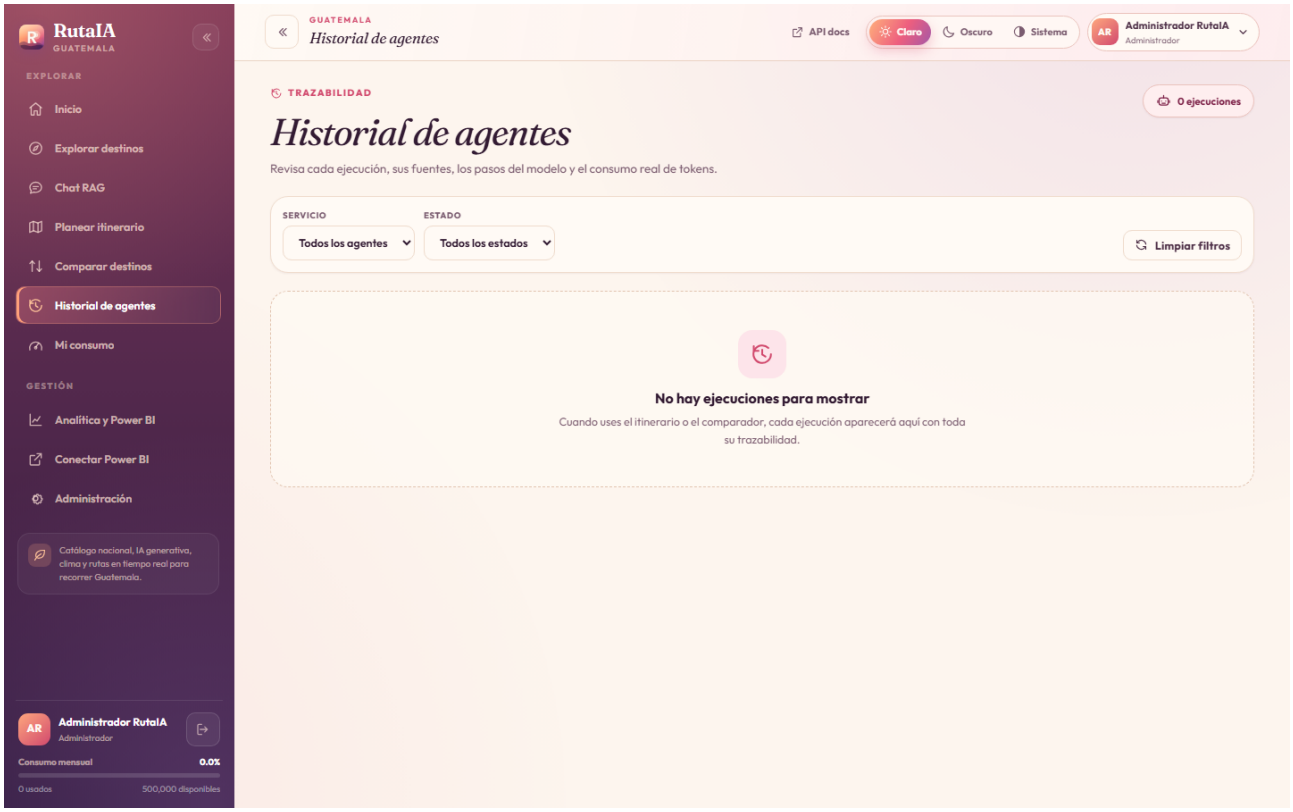
Chat RAG con sugerencias e historial



Agente de itinerarios: formulario e información de herramientas



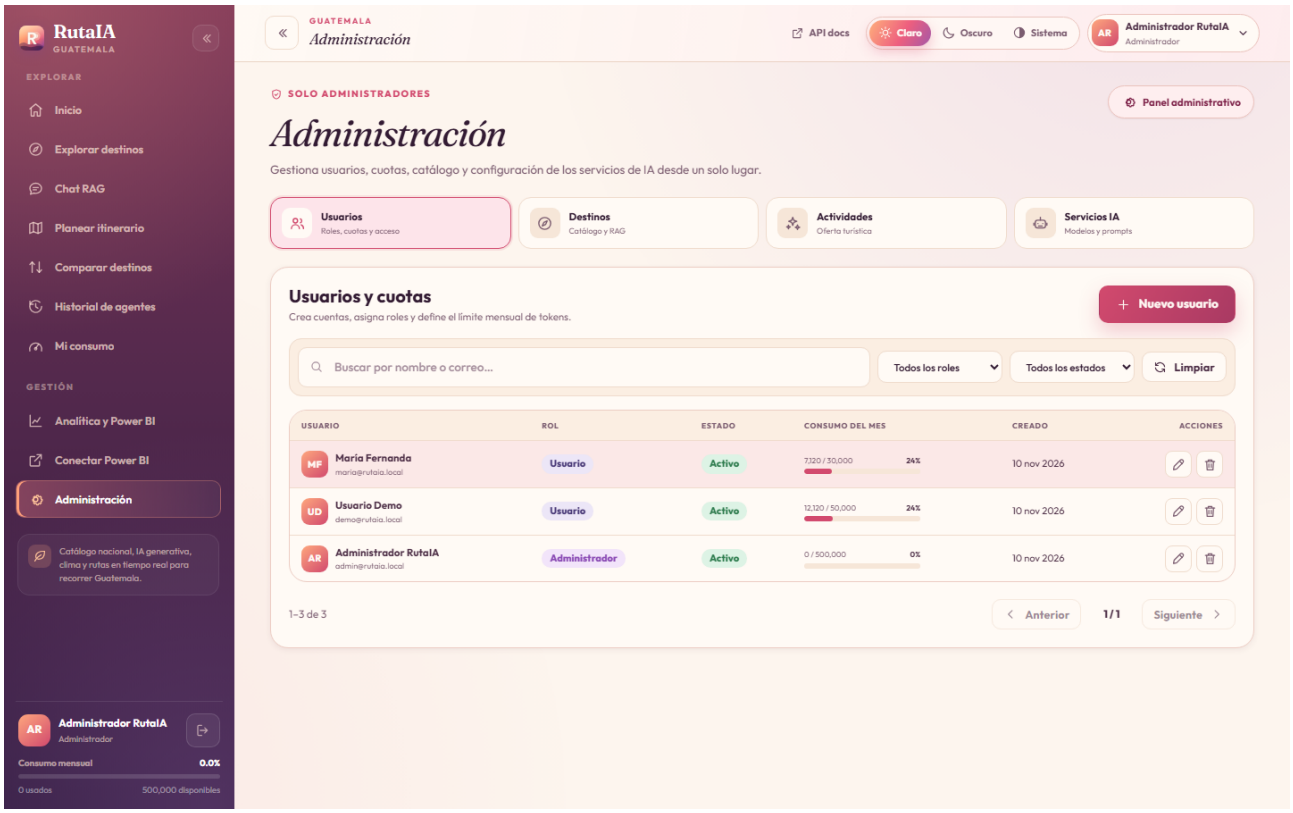
Comparador de destinos con selección múltiple



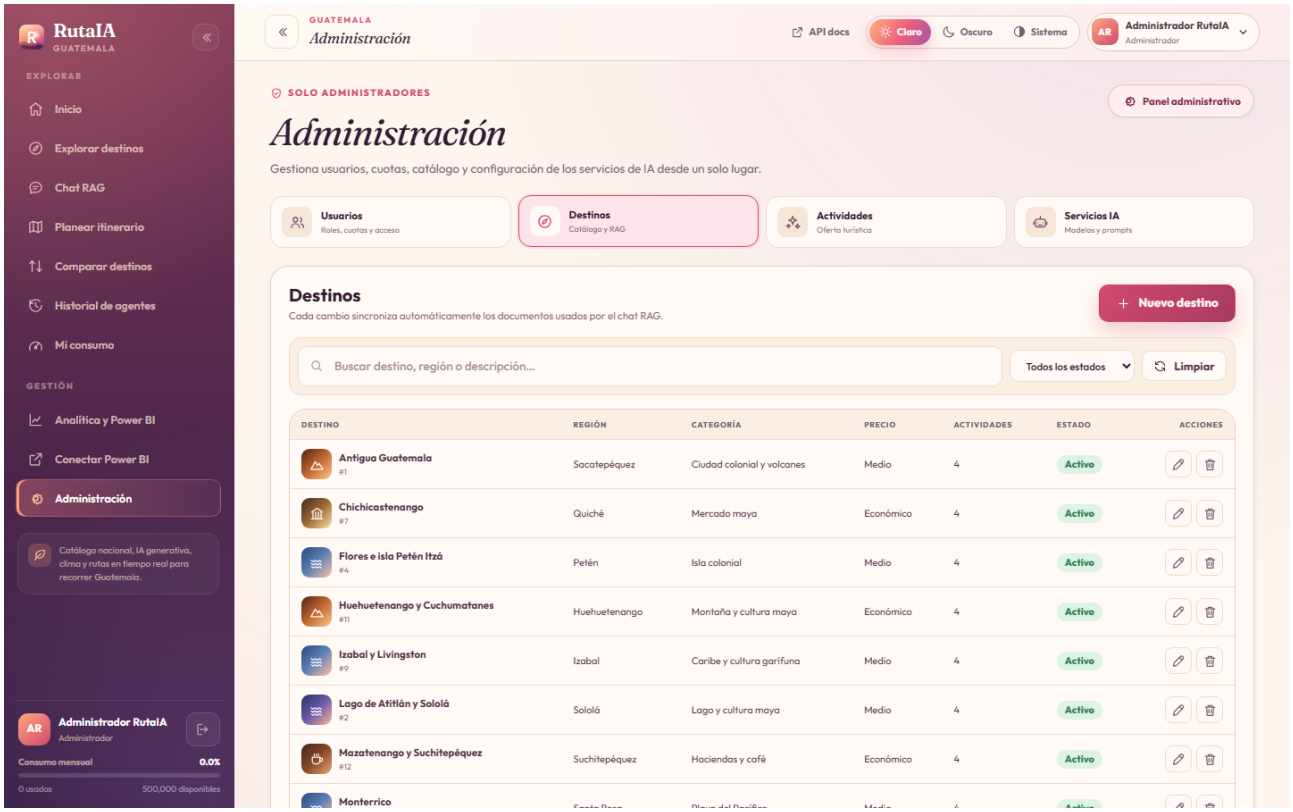
Historial de ejecuciones de agentes



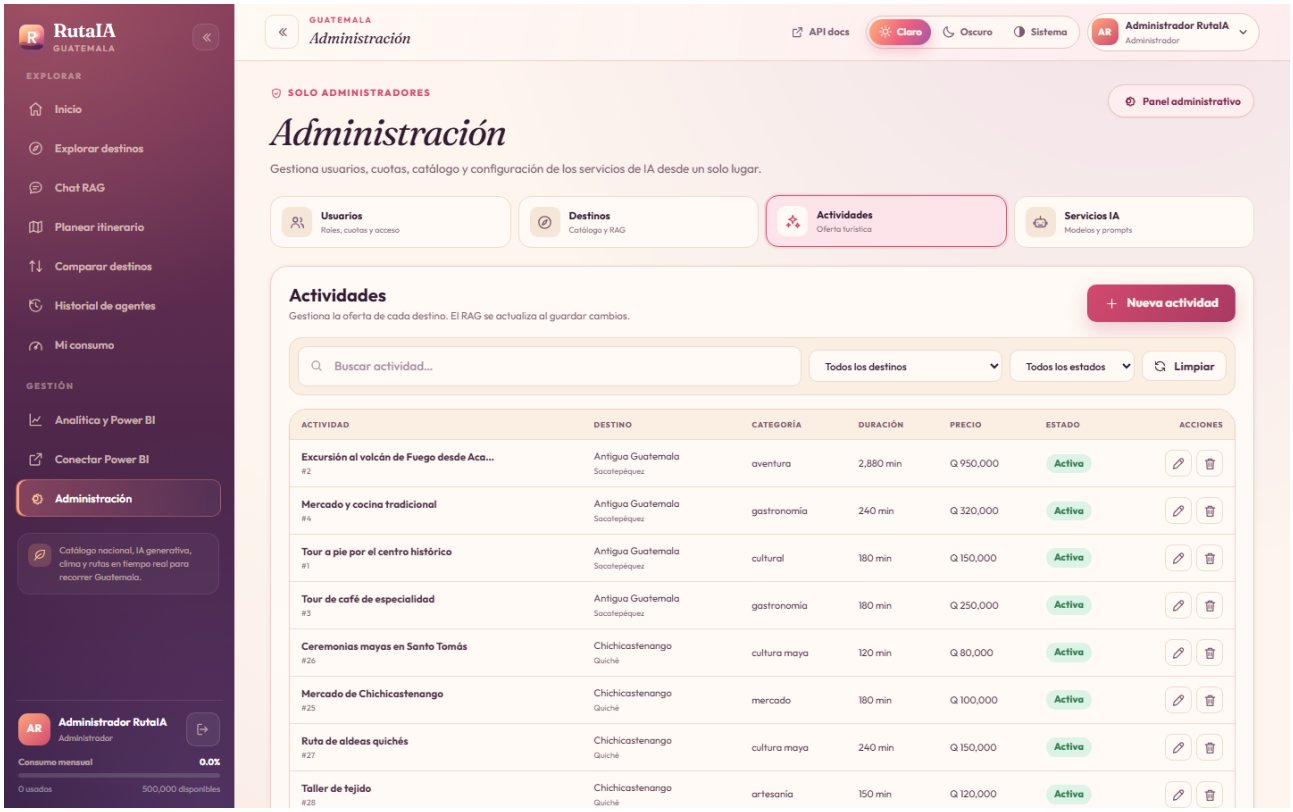
Portal de cuota de tokens



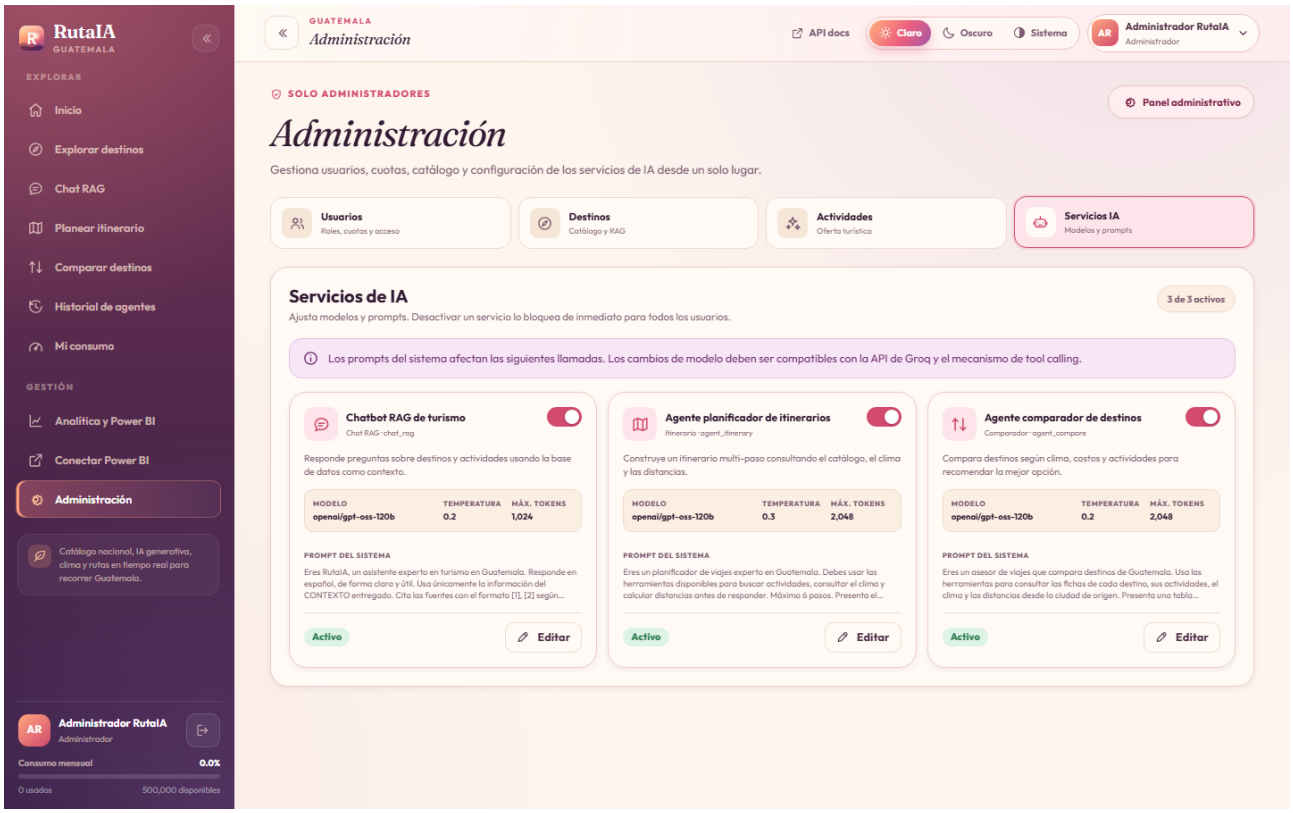
Administración de usuarios y cuotas



Administración de destinos



Administración de actividades

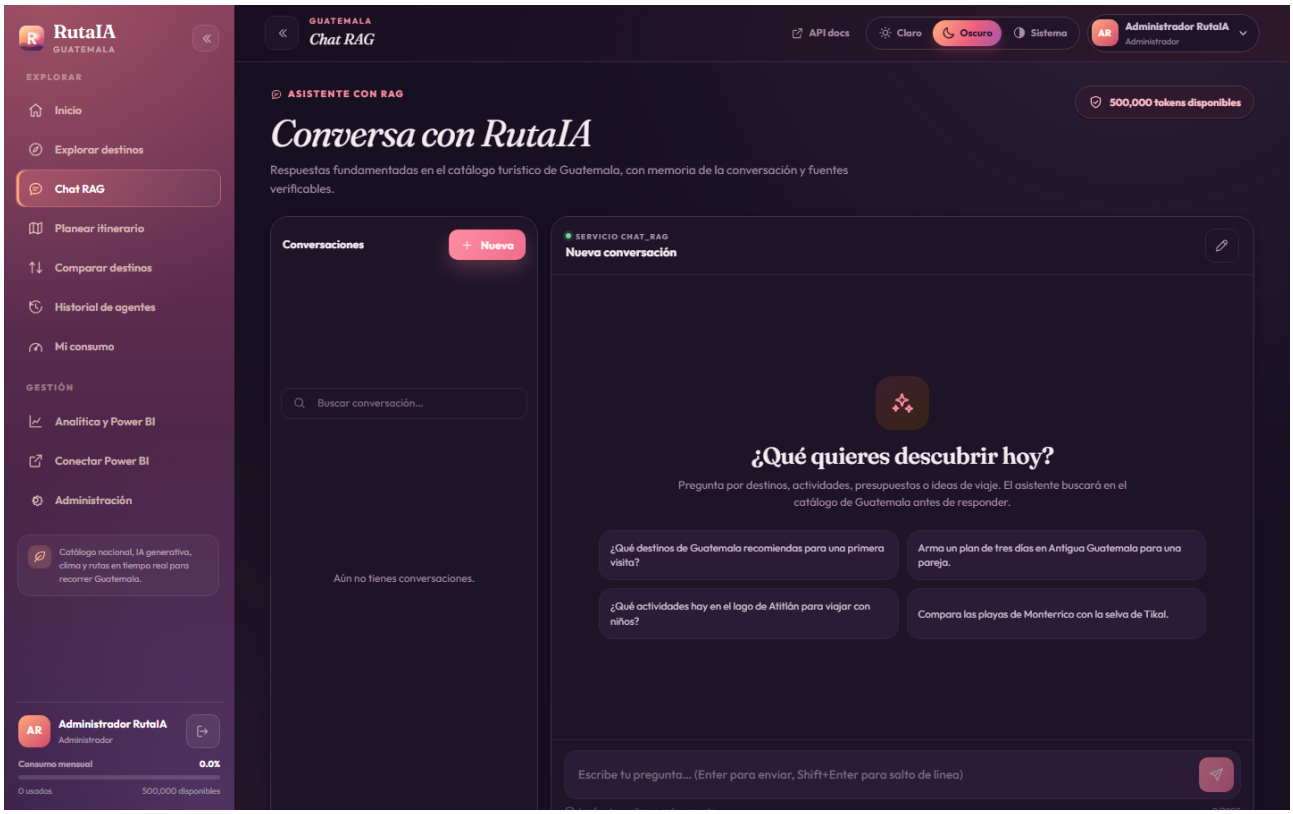


Configuración y activación de servicios de IA

Modo oscuro y vista móvil



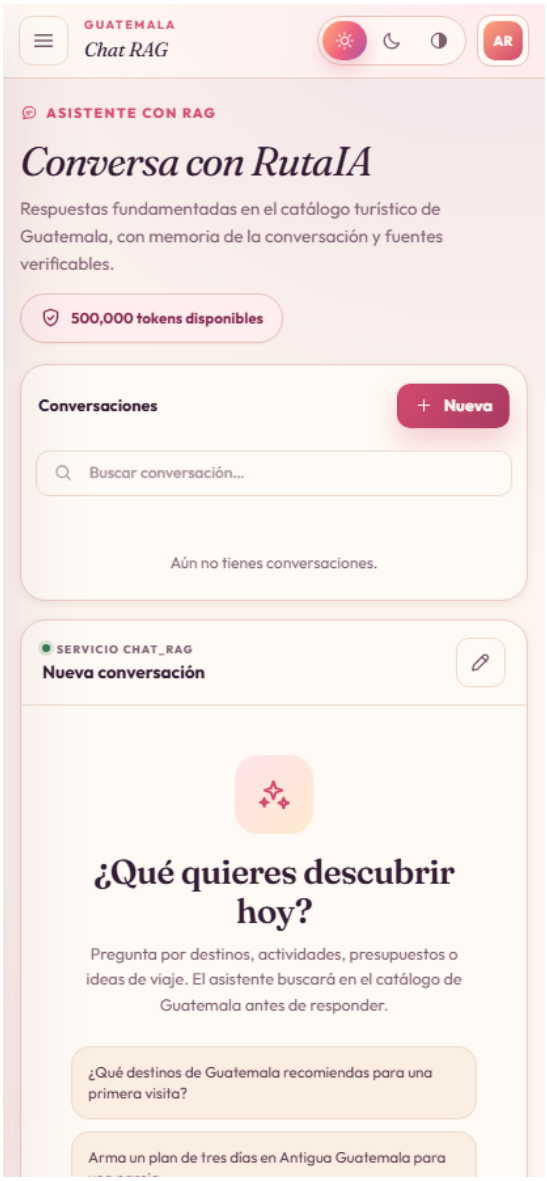
Panel en modo oscuro



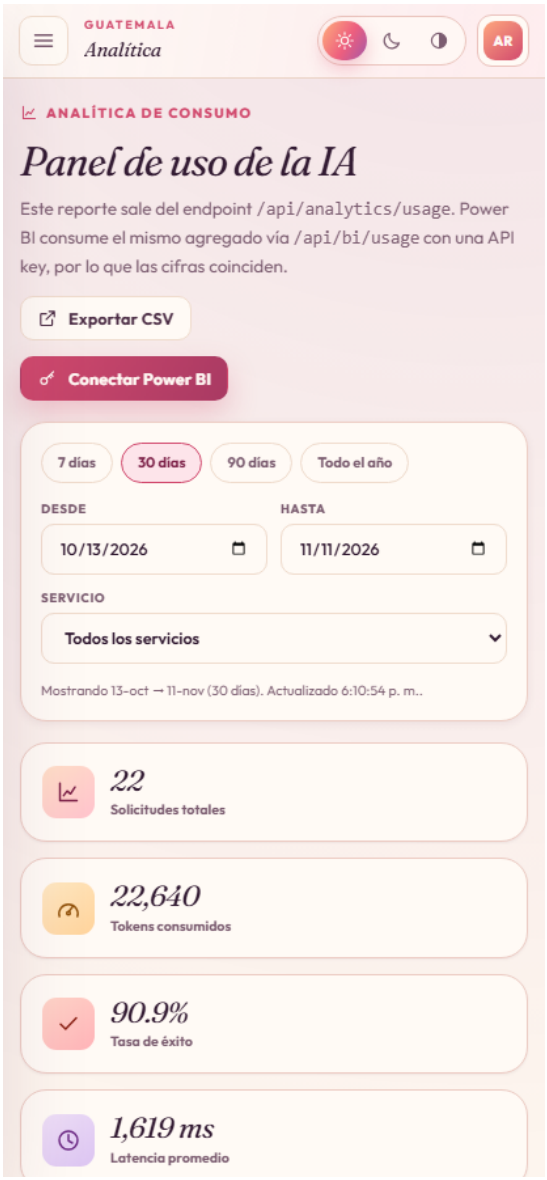
Chat RAG en modo oscuro



Inicio en vista móvil (414 px)



Chat en vista móvil



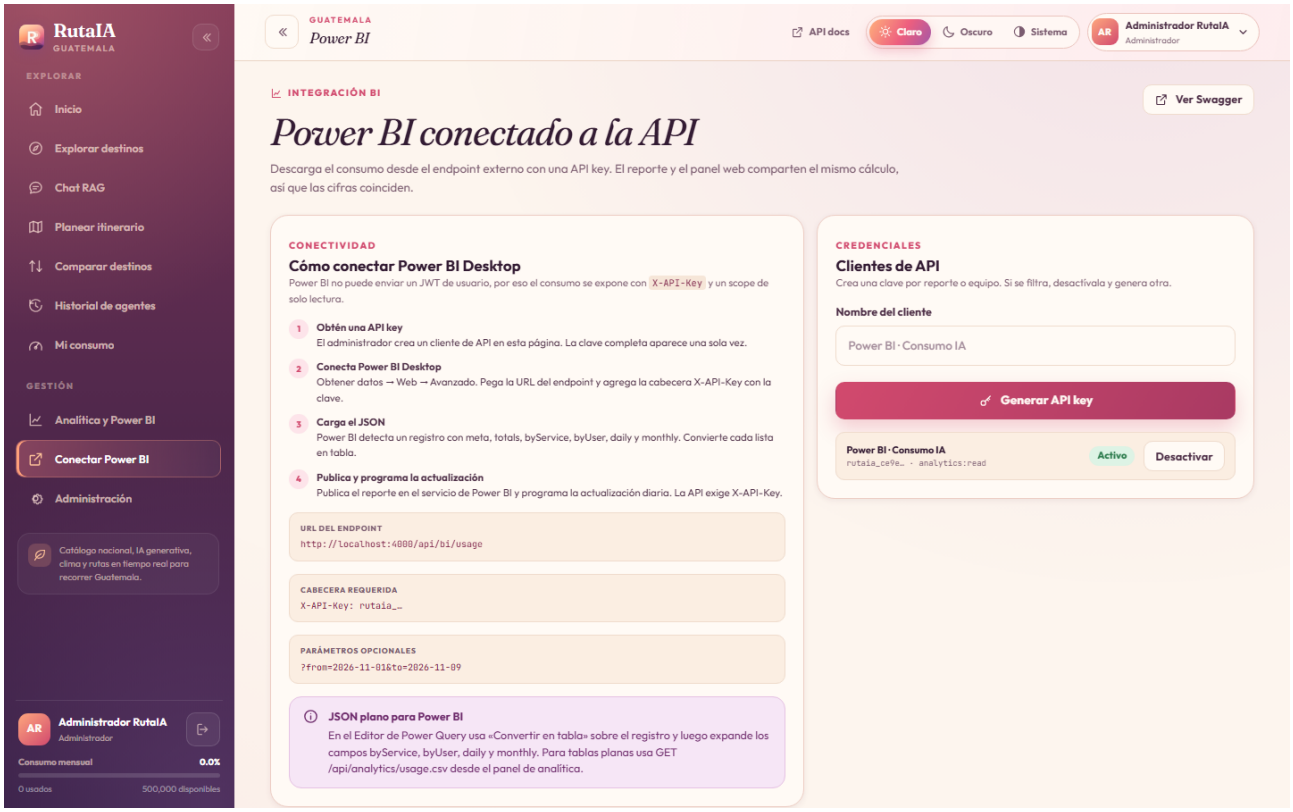
Analítica en vista móvil

6. Analítica y Power BI

El panel web y Power BI consumen la misma función `getUsageAnalytics`. La única diferencia es la autenticación: JWT para el panel y `X-API-Key` para el endpoint externo. El reporte incluye totales, tasa de éxito, latencia, desglose por servicio y por usuario, serie diaria e histórico mensual.



Panel de analítica con series, distribución y tarjetas de resumen



Conexión de Power BI y gestión de API keys

Conexión desde Power BI Desktop

El endpoint devuelve un único documento JSON con seis bloques: `meta`, `totals`, `byService`, `byUser`, `daily` y `monthly`. Los campos son `requests`, `totalTokens`, `serviceCode` y `date`.

```
GET https://rutaia-api-q3y2.onrender.com/api/bi/usage
    ?from=2026-01-01&to=2026-12-31
X-API-Key: rutaia_******************************
```

- Crear la API key en **Conectar Power BI**; solo se muestra una vez.
- Obtener datos → Web → Avanzado, pegar la URL y añadir la cabecera `X-API-Key`.
- Renombrar la consulta base a `BI_Usage` y desactivar su carga (devuelve un registro).
- Crear cinco consultas que la referencien: Totales, Servicios, Usuarios, Diario y Mensual.
- Aplicar y construir los visuales; guardar como `powerbi/reporte_rutaia.pbix`.

El detalle paso a paso, con las consultas M completas y los nombres de campo de RutaIA, está en `powerbi/README.md`.

7. Calidad y pruebas

Las pruebas usan Vitest, Supertest y una base PostgreSQL real. Groq y las APIs externas se simulan con mocks, de modo que la suite es determinista y no consume cuota. Cada archivo crea sus propios usuarios y limpia sus datos.

Archivo	Enfoque	Pruebas
api.test.js	Salud, errores, 404, rate limit y seguridad básica	8
fase3.test.js	Login, roles, usuarios y catálogo administrable	31
fase4.test.js	Chat RAG, memoria, fuentes y cuota	10
fase5.test.js	Agentes, herramientas, trazas y cuotas	10
fase7.test.js	Analítica, CSV, API keys y endpoint externo	10
groqClient.test.js	Contrato del cliente de Groq y recuperación de tool calls	6
Total	Vitest + Supertest sobre PostgreSQL 16	75

Entre las pruebas se incluye una que rechaza coordenadas fuera del *bounding box* de Guatemala, de modo que el alcance geográfico del proyecto está cubierto por la suite y no solo por el texto del prompt.

La CI de GitHub Actions reejecuta las pruebas desde cero contra PostgreSQL 16, aplica esquema y seed, ejecuta la ingesta del RAG y arranca la API. El workflow **smoke.yml** verifica la URL publicada y **codeql.yml** analiza la seguridad del código.

Comandos de verificación

```
cd server && npm test              # 75 pruebas
cd client && npm run build         # build de producción
node scripts/capturas.mjs          # capturas de pantalla
node scripts/smoke-api.mjs <url>   # health, login, catálogo y analítica
```

8. Demostración

La colección `docs/postman_collection.json` agrupa las peticiones en siete carpetas. Los scripts de prueba guardan automáticamente el JWT, la conversación, la ejecución y la API key en variables de colección.

Carpeta	Contenido	Variables que produce
00 Sistema	Health check y OpenAPI	—
01 Autenticación	Login admin/demo, perfil y error 401	adminToken, userToken
02 Usuarios	Listado, alta, cuota y 403	—
03 Catálogo	Destinos, actividades y alta	destinationId
04 Chat RAG	Mensaje, memoria, historial y cuota	conversationId
05 Agentes	Itinerario, comparación, historial y detalle	runId
06 Analítica	Reporte, CSV, API key y reporte externo	apiKey

Guion sugerido para 5 minutos

Tiempo	Contenido
0:00 – 0:40	Login, cambio de tema claro/oscuro y barra lateral colapsable
0:40 – 1:40	Panel: 12 destinos, 48 actividades, 3 servicios de IA
1:40 – 2:30	Catálogo con filtros y ficha de un destino
2:30 – 3:30	Chat RAG con una pregunta real y sus fuentes citadas
3:30 – 4:10	Cliente externo en Postman: GET /api/bi/usage con X-API-Key
4:10 – 4:50	Reporte de Power BI conectado a la API
4:50 – 5:00	Enlaces: repositorio, aplicación y Swagger

9. Despliegue y operación

Componente	Destino	Configuración
Aplicación	Render Web Service	render.yaml · build de cliente y servidor · start node server/src/server.js
Base de datos	PostgreSQL gestionado (Render)	db/schema.postgres.sql + db/seed.postgres.sql + npm run db:ingest
Imagen local	Docker Compose	postgres:16-alpine + perfiles opcionales
CI/CD	GitHub Actions	ci.yml, codeql.yml y smoke.yml

Enlaces de la entrega

Recurso	URL
Aplicación web	https://rutaia-api-q3y2.onrender.com
Swagger / API docs	https://rutaia-api-q3y2.onrender.com/api/docs
OpenAPI (JSON)	https://rutaia-api-q3y2.onrender.com/api/docs.json
Health check	https://rutaia-api-q3y2.onrender.com/api/health
Power BI	https://rutaia-api-q3y2.onrender.com/api/bi/usage (X-API-Key)
Repositorio	https://github.com/FerMar707501/PFinalTurismo

El plan gratuito de Render suspende el servicio tras 15 minutos sin tráfico; la primera petición puede tardar alrededor de 50 segundos en despertarlo. Conviene abrir la aplicación unos minutos antes de la demostración.

Variables de entorno

Variable	Ámbito	Descripción
DATABASE_URL	API	Conexión a PostgreSQL (Render la inyecta)
DATABASE_SSL	API	TLS hacia la base gestionada
JWT_SECRET, JWT_EXPIRES_IN	API	Firma y vigencia del JWT
GROQ_API_KEY, GROQ_MODEL, GROQ_MODEL_LITE	API	Proveedor de IA y modelos
CORS_ORIGIN	API	Orígenes permitidos
TOKEN_QUOTA_DEFAULT	API	Cuota inicial de nuevos usuarios

10. Seguridad, limitaciones y trabajo futuro

- Contraseñas con bcrypt y JWT firmados; el token se valida contra el estado actual del usuario.
- Las API keys se guardan como hash SHA-256 y se muestran una única vez.
- Consultas SQL parametrizadas, validación Zod y límite de intentos de login.
- Helmet con CSP explícita, CORS restringido, rate limit y cabecera X-Request-Id en cada respuesta.
- El system_prompt de los servicios solo se expone a administradores.
- La GROQ_API_KEY vive únicamente en el backend.

Limitaciones conocidas

- La cuota se controla con una reserva estimada; dos solicitudes simultáneas podrían competir por el último margen.
- El RAG usa recuperación léxica con tsvector; no hay embeddings porque Groq no expone esa API.
- Wikipedia puede limitar solicitudes frecuentes; la caché y el manejo de errores de herramientas lo mitigan.
- El plan gratuito de Render suspende la API por inactividad.

Trabajo futuro

- Reserva atómica de cuota dentro de una transacción y conciliación posterior.
- Embeddings con un proveedor dedicado y búsqueda híbrida.
- Panel de auditoría de API keys con rotación programada.
- Reservas y pagos integrados con el catálogo.
- Pruebas end-to-end de navegador en la CI con Playwright.

Documento generado a partir del código y de las capturas verificadas del repositorio. Las cifras de analítica corresponden a la base de demostración, por lo que son reproducibles con el mismo seed.